Data Science Experiment 5 – Feature Selection

Using `urban_climate.csv` to select features for predicting `urban_heat_island_intensity`. The target is continuous, so this is a regression task.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import mutual_info_regression, RFE
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

OUTPUT = Path('outputs')
OUTPUT.mkdir(exist_ok=True)
```

Steps 1–3: Load data, define X/y and clean

In [2]:
```
df = pd.read_csv('urban_climate.csv')
print('Shape:', df.shape)
display(df.head())
print('Missing values:\n', df.isnull().sum())
print('Duplicate rows:', df.duplicated().sum())
TARGET = 'urban_heat_island_intensity'
feature_cols = ['latitude','longitude','year','month','temperature_celsius','hum
X = df[feature_cols].replace([np.inf,-np.inf], np.nan).copy()
y = df[TARGET].copy()
for col in X.columns:
    X[col] = X[col].fillna(X[col].median())
valid = y.notna()
X = X.loc[valid].reset_index(drop=True)
y = y.loc[valid].reset_index(drop=True)
```

```
Shape: (11040, 20)
```

	city	country	latitude	longitude	year	month	temperature_celsius	humidity_percen
0	New York	USA	40.7128	-74.006	1980	1	6.21	48.
1	New York	USA	40.7128	-74.006	1980	2	17.46	38.
2	New York	USA	40.7128	-74.006	1980	3	20.28	32.
3	New York	USA	40.7128	-74.006	1980	4	22.31	30.
4	New York	USA	40.7128	-74.006	1980	5	20.01	22.

```
Missing values:
 city                            0
country                          0
latitude                         0
longitude                        0
year                             0
month                            0
temperature_celsius              0
humidity_percent                 0
precipitation_mm                 0
wind_speed_ms                    0
urban_heat_island_intensity      0
data_source                      0
country_population_reference     0
country_area_km2                 0
continent                        0
subregion                        0
capital_city                     0
calling_code                     0
iso3_code                        0
currency                         0
dtype: int64
Duplicate rows: 0
```

Train-test split

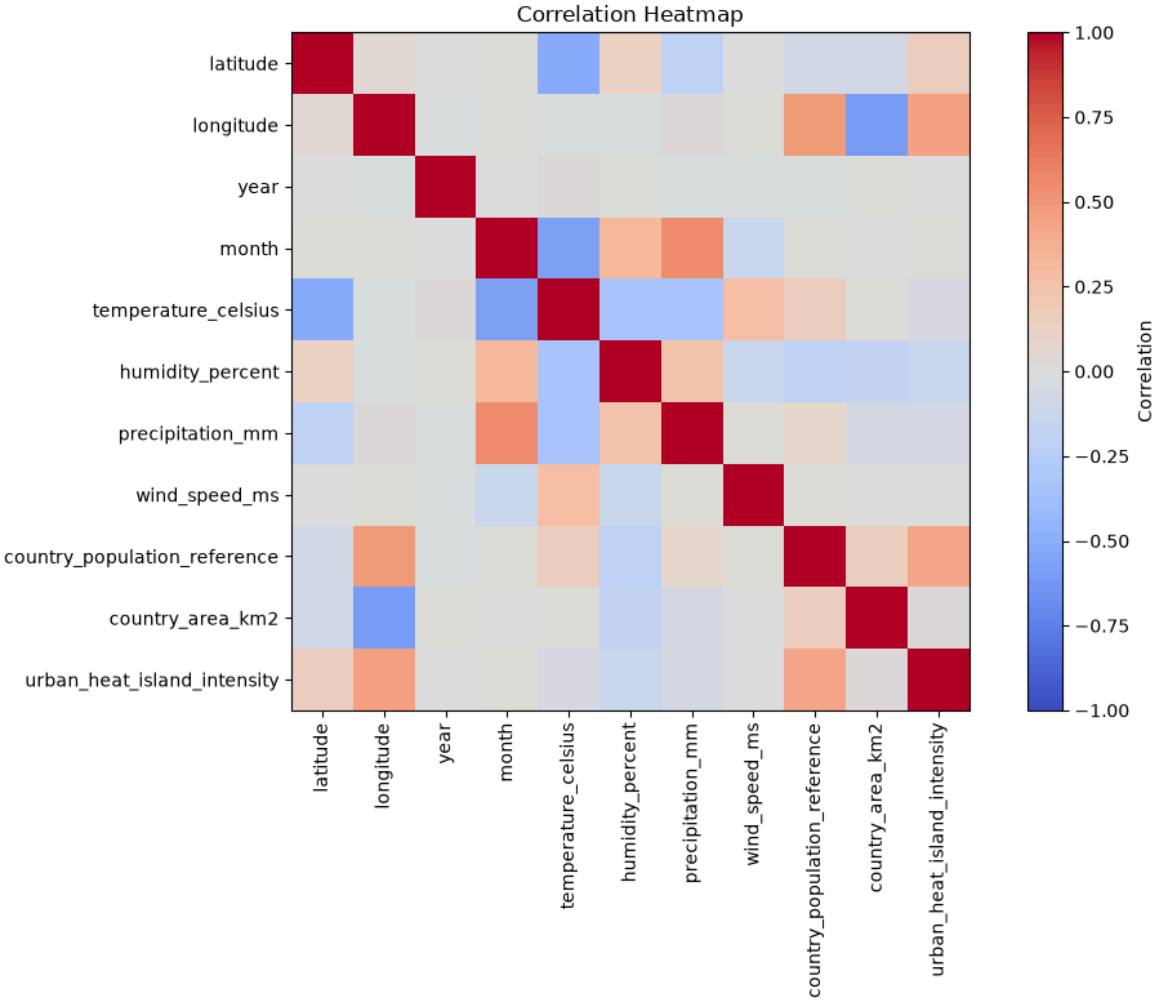
In [3]:
```
train_idx, test_idx = train_test_split(np.arange(len(X)), test_size=0.2, random_
X_train, X_test = X.iloc[train_idx].copy(), X.iloc[test_idx].copy()
y_train, y_test = y.iloc[train_idx].copy(), y.iloc[test_idx].copy()
```

Step 4: Correlation analysis

In [4]:
```
train_corr = X_train.copy()
train_corr[TARGET] = y_train.values
corr_matrix = train_corr.corr(numeric_only=True)
corr_rank = corr_matrix[TARGET].drop(TARGET).abs().sort_values(ascending=False)
display(corr_rank)
plt.figure(figsize=(11,8)); plt.imshow(corr_matrix, cmap='coolwarm', vmin=-1, vm
```

```
plt.xticks(range(len(corr_matrix.columns)), corr_matrix.columns, rotation=90); p
plt.title('Correlation Heatmap'); plt.tight_layout(); plt.savefig(OUTPUT/'01_cor
```

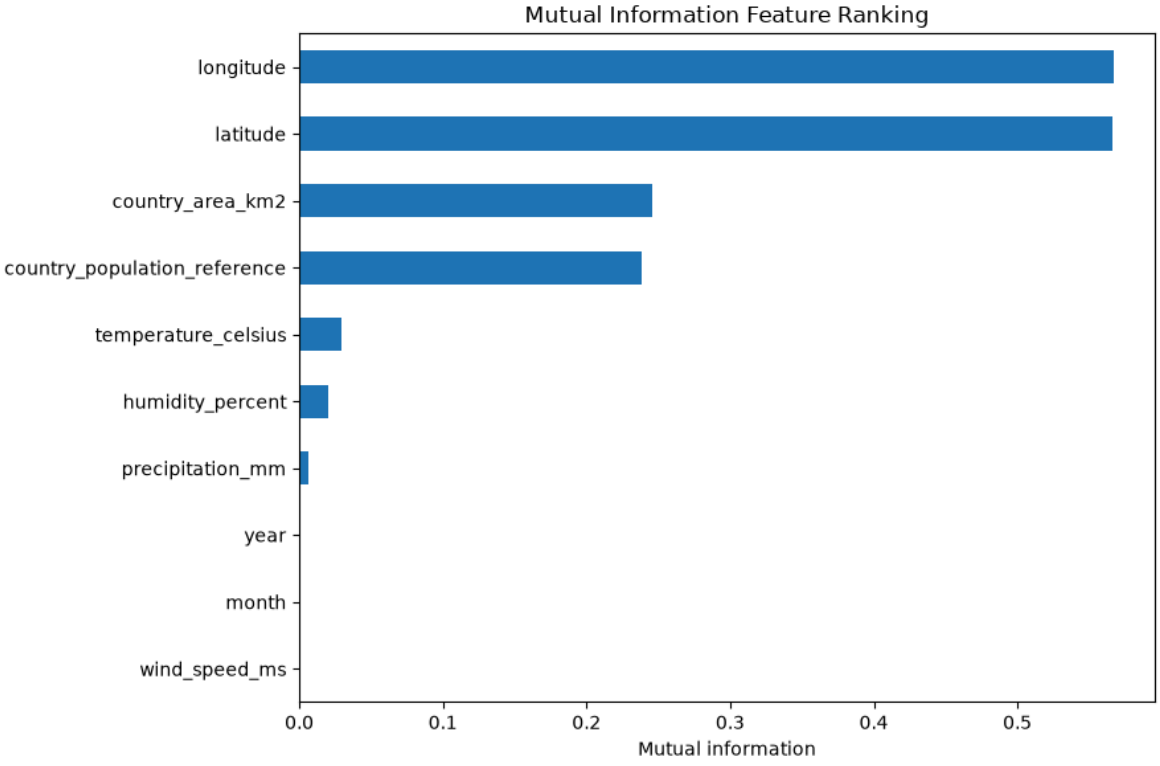
```
longitude                       0.456760
country_population_reference    0.415094
latitude                        0.147378
humidity_percent                0.124806
precipitation_mm                0.054208
temperature_celsius             0.035013
country_area_km2                0.030289
month                           0.005679
year                            0.004332
wind_speed_ms                   0.003938
Name: urban_heat_island_intensity, dtype: float64
```



Step 5: Filter method – Mutual Information

In [5]:
```
mi_values = mutual_info_regression(X_train, y_train, random_state=42)
mi_rank = pd.Series(mi_values, index=feature_cols).sort_values(ascending=False)
display(mi_rank)
mi_rank.sort_values().plot(kind='barh', figsize=(9,6), title='Mutual Information
plt.xlabel('Mutual information'); plt.tight_layout(); plt.savefig(OUTPUT/'02_mut
```

```
longitude                     0.567305
latitude                      0.565928
country_area_km2              0.246079
country_population_reference  0.238673
temperature_celsius           0.029537
humidity_percent              0.020113
precipitation_mm              0.006744
year                          0.000958
month                         0.000000
wind_speed_ms                 0.000000
dtype: float64
```



Step 6: Wrapper method – Recursive Feature Elimination

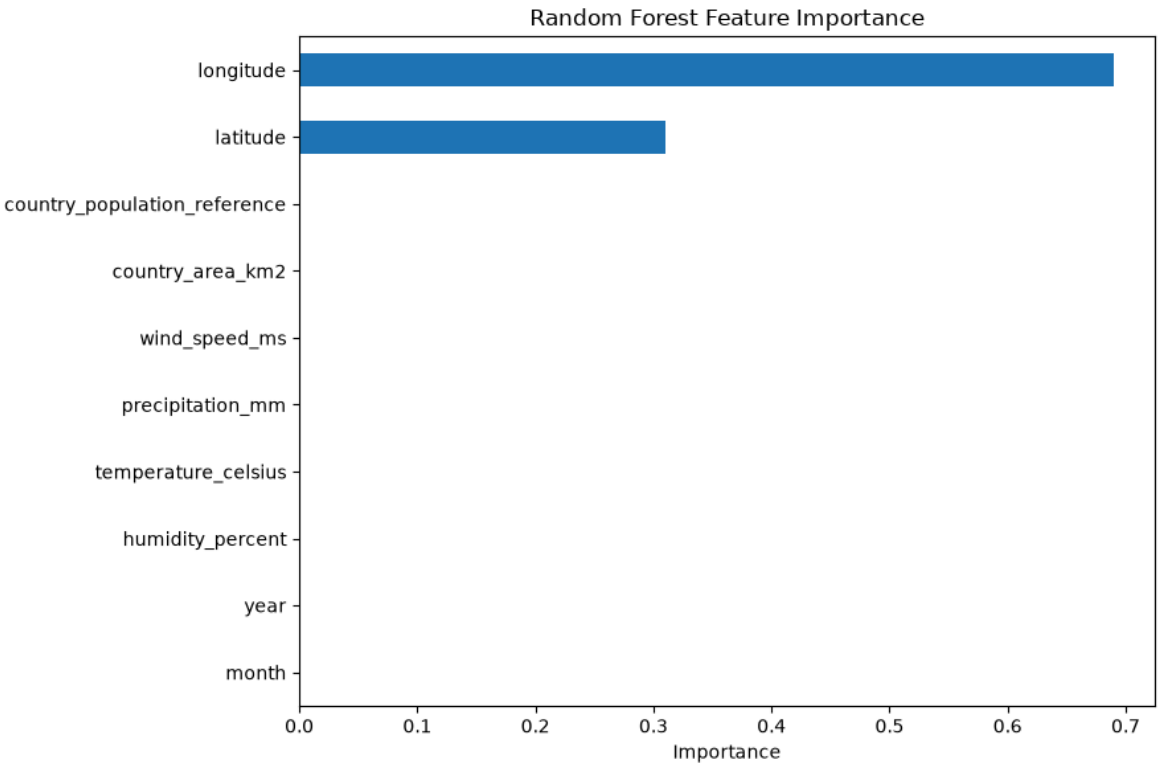
In [6]:
```
rfe = RFE(RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1), n
rfe.fit(X_train, y_train)
rfe_features = X_train.columns[rfe.support_].tolist()
rfe_rank = pd.DataFrame({'Feature':feature_cols, 'Selected':rfe.support_, 'RFE_R
display(rfe_rank)
rfe_rank.to_csv(OUTPUT/'rfe_feature_ranking.csv', index=False)
```

	Feature	Selected	RFE_Rank
0	latitude	True	1
1	longitude	True	1
7	wind_speed_ms	True	1
8	country_population_reference	True	1
9	country_area_km2	True	1
6	precipitation_mm	False	2
5	humidity_percent	False	3
4	temperature_celsius	False	4
3	month	False	5
2	year	False	6

Step 7: Embedded method – Random Forest feature importance

In [7]:
```
embedded_model = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs
embedded_model.fit(X_train, y_train)
importance = pd.Series(embedded_model.feature_importances_, index=feature_cols).
display(importance)
importance.sort_values().plot(kind='barh', figsize=(9,6), title='Random Forest F
plt.xlabel('Importance'); plt.tight_layout(); plt.savefig(OUTPUT/'03_random_fore
```

```
longitude                      0.690143
latitude                       0.309857
year                           0.000000
month                          0.000000
temperature_celsius            0.000000
humidity_percent               0.000000
precipitation_mm               0.000000
wind_speed_ms                  0.000000
country_population_reference   0.000000
country_area_km2               0.000000
dtype: float64
```



Step 8: Compare selected features

In [8]:
```
comparison = pd.DataFrame({'Feature':feature_cols, 'Correlation_Top5':[f in corr
comparison['Selection_Count'] = comparison.iloc[:,1:5].sum(axis=1)
comparison = comparison.sort_values(['Selection_Count','Feature'], ascending=[Fa
display(comparison)
comparison.to_csv(OUTPUT/'feature_selection_comparison.csv', index=False)
selected_features = rfe_features
print('RFE-selected features:', selected_features)
```

	Feature	Correlation_Top5	Mutual_Info_Top5	RFE_Selected	Random
0	latitude	True	True	True	
1	longitude	True	True	True	
8	country_population_reference	True	True	True	
9	country_area_km2	False	True	True	
4	temperature_celsius	False	True	False	
5	humidity_percent	True	False	False	
3	month	False	False	False	
6	precipitation_mm	True	False	False	
7	wind_speed_ms	False	False	True	
2	year	False	False	False	

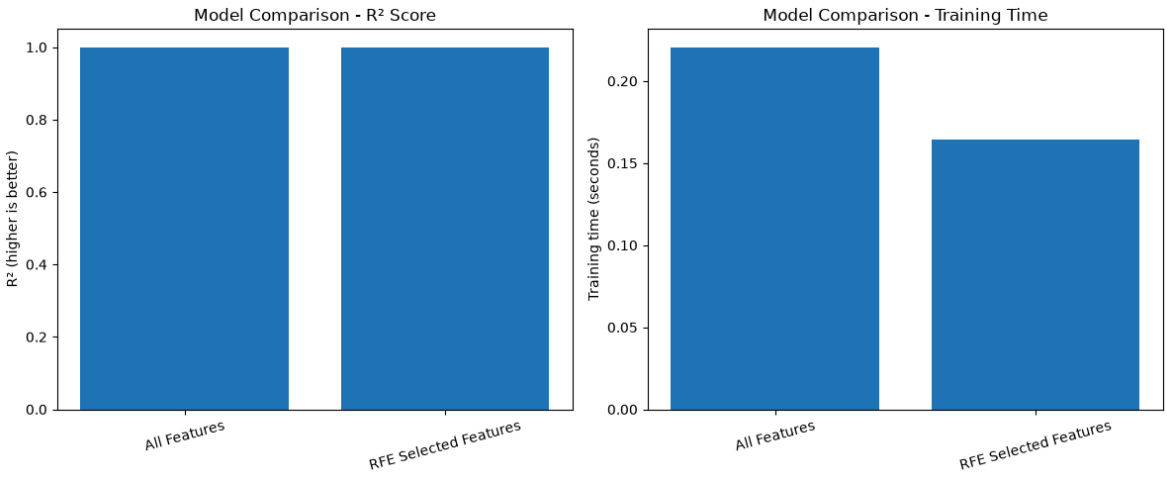
```
RFE-selected features: ['latitude', 'longitude', 'wind_speed_ms', 'country_popula
tion_reference', 'country_area_km2']
```

Steps 9–11: Train and compare models

In [9]:
```
def evaluate_model(model_features, model_name):
    model = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)
    start = time.perf_counter(); model.fit(X_train[model_features], y_train); tr
    start = time.perf_counter(); pred = model.predict(X_test[model_features]); p
    return {'Model':model_name, 'Number_of_Features':len(model_features), 'R2_Sc
results = pd.DataFrame([evaluate_model(feature_cols,'All Features'), evaluate_mo
display(results)
results.to_csv(OUTPUT/'model_performance_comparison.csv', index=False)
```

	Model	Number_of_Features	R2_Score	MAE	RMSE	Training_Time_Seconds	Predicti
0	All Features	10	1.0	0.0	0.0	0.220653	
1	RFE Selected Features	5	1.0	0.0	0.0	0.164071	

In [10]:
```
# Model comparison graph: predictive performance and computational time
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].bar(results['Model'], results['R2_Score'])
axes[0].set_title('Model Comparison - R² Score')
axes[0].set_ylabel('R² (higher is better)')
axes[0].tick_params(axis='x', rotation=15)
axes[1].bar(results['Model'], results['Training_Time_Seconds'])
axes[1].set_title('Model Comparison - Training Time')
axes[1].set_ylabel('Training time (seconds)')
axes[1].tick_params(axis='x', rotation=15)
plt.tight_layout()
plt.savefig(OUTPUT/'04_model_comparison_graph.png', dpi=160)
plt.show()
```



Final interpretation

In [11]:
```
print('Target:', TARGET)
print('Original features:', len(feature_cols))
```

```
print('Selected features:', selected_features)
print('Compare R2 (higher is better), MAE/RMSE (lower is better), and execution
```

```
Target: urban_heat_island_intensity
Original features: 10
Selected features: ['latitude', 'longitude', 'wind_speed_ms', 'country_population
_reference', 'country_area_km2']
Compare R2 (higher is better), MAE/RMSE (lower is better), and execution times.
```